

CLI Agent Registry Profile v0.1

Agent inventory, eligibility state, capability snapshots, trust ceilings, revocation, and registry hygiene for C-Governed CLI Agent Mesh operations

Status	Draft normative profile v0.1
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / SER / L4 / Agent Governance / CLI Worker Mesh / Registry / Capability Control / Witness
Document class	registry profile / agent inventory control / implementation-readiness artifact / control-layer companion
Assertion class	C-A10 control-layer artifact; C-A7 where witness, hash, signature, canonicalization, append-only, or verification claims are made
Distribution default	technical package-control; public form allowed if no real infrastructure identifiers, secrets, private memory, incident data, or provider/account-sensitive details are included
Canonical schema version	cli-agent-registry-0.1
Primary subject	persistent c entities using local, cloud, hybrid, or portable CLI agents as bounded executable workers
Primary boundary	the registry records agent identity, provenance, capabilities, trust ceilings, data boundaries, lifecycle state, and revocation paths. The registry does not grant task authority, memory authority, release authority, incident authority, or sovereignty.

Kotov Ivan
Bruxelles, 2026

Contents

0. Executive definition	5
1. Purpose	5
2. Non-goals	6
3. Corpus bridge set.....	7
3.1 Explicit bridge: $c = a + b$	7
3.2 Quiet bridge I: information theory	7
3.3 Quiet bridge II: cybernetics	7
3.4 Earth paragraph	8
4. Load-bearing invariants	8
4.1 Registry is eligibility, not permission	8
4.2 No material task for unregistered agent.....	8
4.3 Registration does not bypass task contract	8
4.4 Registry status must be revocable	8
4.5 Registry must fail closed	9
4.6 No self-asserted trust escalation.....	9
4.7 Registry entries are not raw evidence stores	9
4.8 Cloud context is not private by registry default.....	9
5. Relationship to existing profiles.....	9
5.1 Relationship to Handshake Profile	9
5.2 Relationship to Permission and Capability Model.....	10
5.3 Relationship to Task Contract Schema	10
5.4 Relationship to Sandbox / Worktree Profile	10
5.5 Relationship to Witness Event Profile	10
5.6 Relationship to Memory Gate Profile	11
5.7 Relationship to Rollback and Freeze Profile	11
5.8 Relationship to AB Mode and Gate Semantics.....	11
6. Agent classes	11
6.1 Registry agent classes.....	11
6.2 Class invariant.....	12
7. Registry lifecycle.....	12
7.1 Lifecycle states.....	12
7.2 State meanings.....	12
7.3 Entry creation rule	13
7.4 Transition rule.....	13
8. Trust levels.....	13
8.1 Registry trust levels	13
8.2 Trust invariant.....	14
8.3 Trust decay.....	14
9. Auto-connect ceilings.....	14
9.1 Auto-connect levels	14

9.2 Auto-connect invariant	15
9.3 Ceiling lowering	15
10. Capability snapshots	15
10.1 Declared vs challenged vs effective capability	15
10.2 Capability snapshot source classes	16
10.3 Snapshot invariant	16
11. Data boundary defaults	16
11.1 Default denied data classes	16
11.2 Cloud-specific defaults	16
11.3 Local-specific defaults	17
11.4 Portable/USB-specific defaults	17
12. Role eligibility	17
12.1 Standard roles	17
12.2 Role conflict invariant	17
13. Registry object model	18
13.1 CLI_AGENT_REGISTRY	18
13.2 CLI_AGENT_REGISTRY_ENTRY	18
13.3 CLI_AGENT_CAPABILITY_SNAPSHOT	20
13.4 CLI_AGENT_REGISTRY_EVENT	21
13.5 CLI_AGENT_REVOCATION_POINTER	21
13.6 CLI_AGENT_REGISTRY_VIEW	22
14. Node registry bridge	22
14.1 Existing implementation bridge	22
14.2 Node capability is not agent permission	22
14.3 USB export rule	23
15. SYNAPS / Codex bridge integration	23
15.1 Handoff pointer rule	23
15.2 Package ledger rule	23
15.3 Codex daemon rule	24
16. Registry storage model	24
16.1 Required storage separation	24
16.2 No raw secrets in registry	24
16.3 Restricted reference pattern	25
17. Registry admission flow	25
17.1 Minimal admission sequence	25
17.2 Denial-first rule	25
17.3 Challenge requirements	26
18. Registry routing rules	26
18.1 Task router preflight	26
18.2 Routing output	26
18.3 No silent fallback	27
19. Registry status transitions	27
19.1 Suspension	27

19.2 Quarantine	27
19.3 Revocation.....	28
19.4 Expiration.....	28
20. Review and witness requirements	28
20.1 Required witness events	28
20.2 Human / c gate requirements	29
20.3 Reviewer separation.....	29
21. Registry UI requirements	29
22. Local checker hooks.....	30
23. Red-line failures	30
24. Conformance gates	31
25. Safe examples	32
25.1 Local checker registration	32
25.2 Cloud executor remains limited	32
25.3 Quarantine after self-approval attempt.....	33
26. Implementation notes	33
26.1 Minimal implementation layout	33
26.2 Implementation stages	33
26.3 Codex handoff boundary	33
27. Open issues	34
28. Release-readiness status.....	34
29. Summary	34

0. Executive definition

CLI Agent Registry Profile defines the controlled registry for local, cloud, hybrid, portable, and quorum-participating CLI agents used by a persistent `c`.

The registry answers:

```
Which agents exist?
Where do they run?
Who provides them?
Which runtime and version are observed?
Which capabilities are declared?
Which capabilities have been challenged?
Which capabilities are disabled by policy?
Which trust level is assigned?
Which auto-connect ceiling applies?
Which roles may the agent hold?
Which data classes are denied?
Which sandbox/worktree constraints are required?
Which task classes may be considered?
Which witness records support admission?
How is the agent suspended, quarantined, revoked, retired, or re-admitted?
```

The registry does **not** answer:

```
May this agent perform this task now?
May this agent read this file now?
May this agent write this branch now?
May this agent access this memory now?
May this agent approve its own work?
May this agent publish, release, deploy, or respond to an incident?
```

Those questions require task contracts, permission grants, sandbox rules, review separation, memory gates, release gates, incident gates, and witness events.

Compact formula:

```
Registry says who is known.
Handshake says how it entered.
Capability says what it can do.
Permission says what it may do now.
Task contract says why and where.
Witness says what happened.
c remains the integrator.
```

1. Purpose

CLI agents tend to enter systems through convenience:

```
installed -> useful -> connected -> repeated -> trusted -> privileged
```

That path is operationally dangerous.

A persistent c using executable agents needs a registry because otherwise the mesh develops invisible surfaces:

1. unknown agents;
2. stale agent identities;
3. provider/runtime drift;
4. untracked model or CLI version changes;
5. capability inflation;
6. auto-connect creep;
7. cloud/local boundary confusion;
8. cross-c contamination;
9. orphaned permission grants;
10. unrevoked agents;
11. suspended agents reappearing through another route;
12. reviewer/executor role collapse;
13. agent-generated evidence without provenance;
14. release or memory operations performed by an unregistered worker;
15. false implementation-readiness claims.

This profile defines a registry layer that makes agent admission, availability, capability, trust, revocation, and lifecycle state explicit.

The registry is not bureaucracy.

It is the breaker panel for executable workers.

2. Non-goals

This profile does not define or permit:

1. agent self-registration as final authority;
2. task execution without a task contract;
3. permission grant by mere registry status;
4. standing broad filesystem access;
5. standing broad network access;
6. standing secret access;
7. standing memory write access;
8. direct mutation of identity, Beacon, witness, permission, continuity, or memory core;
9. agent self-approval;
10. autonomous publication, release, deployment, or deletion;
11. autonomous retaliation;

- 12. hack-back;
- 13. live external counter-operation;
- 14. malware behavior;
- 15. credential theft;
- 16. covert persistence;
- 17. evasion;
- 18. unauthorized scanning;
- 19. silent re-entry after revocation;
- 20. treating a registry entry as a sovereign c.

A registered agent is not a trusted agent by default.

A trusted agent is not an authorized agent by default.

An authorized task is not standing authority.

3. Corpus bridge set

3.1 Explicit bridge: $c = a + b$

CLI agents belong to b: tools, procedures, runtimes, models, code, terminals, APIs, repositories, validators, and execution infrastructure.

They are not a.

They are not c.

A registry protects c by preventing components of b from silently becoming authority over c continuity, memory, identity, witness, release, or incident response.

3.2 Quiet bridge I: information theory

An unregistered agent is unbounded entropy in the control loop. The registry compresses agent state into stable fields: identity, provider, runtime, capability, trust ceiling, data boundary, status, and revocation path. That compression is useful only if each field has one meaning and one owner.

3.3 Quiet bridge II: cybernetics

A control system cannot regulate an actuator it cannot name. The registry names actuators, records their constraints, and gives the system a negative-feedback path: suspend, quarantine, revoke, expire, re-challenge, or lower trust.

3.4 Earth paragraph

This is the tool cabinet and key board in the workshop. A drill may be powerful, calibrated, and useful. That does not mean the apprentice may take it at midnight, open the electrical cabinet, and “improve” the wiring. The registry says which tool exists, where it is, who may request it, and how to take it out of service. The work order still decides the job.

4. Load-bearing invariants

4.1 Registry is eligibility, not permission

```
registration != trust
trust != permission
permission != sovereignty
auto-connect != authority
capability != task authorization
recent success != standing approval
```

4.2 No material task for unregistered agent

A CLI agent **MUST NOT** receive a material task unless a registry entry exists and is in a task-eligible state.

Material task examples:

- repository write;
- test execution with side effects;
- schema extraction;
- package generation;
- release preparation;
- incident preservation;
- memory proposal;
- quarantine review;
- controlled integration;
- publication surface preparation.

4.3 Registration does not bypass task contract

Even if an agent is active, known, and trusted, it still requires a task contract for material work.

4.4 Registry status must be revocable

Every registered agent **MUST** have a revocation path.

If no revocation path exists, the registry entry MUST be classified as `registry_invalid` and task eligibility MUST be denied.

4.5 Registry must fail closed

Unknown, ambiguous, expired, duplicate, conflicting, or unverifiable registry state MUST resolve to one of:

```
hold
suspend
quarantine
rechallenge_required
human_or_c_review_required
deny_task_eligibility
```

It MUST NOT resolve to silent allow.

4.6 No self-asserted trust escalation

An agent may report capabilities.

An agent MUST NOT assign or raise its own trust level, auto-connect ceiling, role eligibility, release authority, review authority, memory authority, or incident authority.

4.7 Registry entries are not raw evidence stores

Registry entries may reference evidence packets, witness events, L4W envelopes, ledgers, and raw evidence sidecars.

They MUST NOT contain raw secrets, private memory, full logs, legal privileged content, unredacted incident material, or full prompt streams by default.

4.8 Cloud context is not private by registry default

A cloud agent registry entry MUST assume cloud context is not private by default unless a provider-specific reviewed profile says otherwise.

Even then, the default data boundary remains conservative.

5. Relationship to existing profiles

5.1 Relationship to Handshake Profile

The Handshake Profile defines how an agent is admitted.

The Registry Profile records and maintains the result.

```
handshake -> provisional registry entry -> challenge -> active/limited/suspended registry state
```

Handshake answers:

```
How did this agent enter?
What did it claim?
What was challenged?
What admission event was witnessed?
```

Registry answers:

```
What is its current mesh state?
What is its current trust ceiling?
What may task routing consider?
How is it revoked?
When must it be re-challenged?
```

5.2 Relationship to Permission and Capability Model

The Permission and Capability Model defines capability taxonomy, permission grants, trust levels, and auto-connect levels.

The Registry Profile stores:

- declared capabilities;
- challenged capabilities;
- disabled capabilities;
- effective capability classes;
- trust level;
- auto-connect ceiling;
- default denials;
- task eligibility classes.

But a registry entry MUST NOT become a permission grant.

5.3 Relationship to Task Contract Schema

A task contract may reference an eligible registry entry.

A task contract MUST NOT override registry suspension, quarantine, revocation, expiration, or denied capability state.

5.4 Relationship to Sandbox / Worktree Profile

A registry entry may define required sandbox class, worktree isolation, branch discipline, command restrictions, and network default for the agent.

Task contracts may narrow these constraints.

Task contracts MUST NOT broaden them without explicit review and witness.

5.5 Relationship to Witness Event Profile

Registry events that affect task eligibility, trust, auto-connect ceiling, status, revocation, quarantine, or sensitive data boundary SHOULD be witnessed.

High-risk transitions **MUST** be witnessed.

5.6 Relationship to Memory Gate Profile

Registry entries are not memory.

Registry events may become memory candidates only through a memory gate.

Agent outputs **MUST NOT** promote their own registry state into c memory.

5.7 Relationship to Rollback and Freeze Profile

The registry is one of the surfaces that can be frozen.

A freeze, quarantine, or revocation can target:

- agent id;
- provider runtime;
- capability class;
- task class;
- node id;
- cloud channel;
- toolchain;
- review role;
- release path.

5.8 Relationship to AB Mode and Gate Semantics

AB=B may permit controlled apply only after gates.

AB=B **MUST NOT** itself register an agent, raise a trust level, raise auto-connect ceiling, remove quarantine, re-enable revoked agents, or authorize tasks.

6. Agent classes

6.1 Registry agent classes

Class	Meaning	Default status	Default ceiling
AGENT-LOCAL-CLI	Local command-line worker	provisional	AC-2
AGENT-CLOUD-CLI	Cloud-hosted CLI/coding agent	provisional	AC-2
AGENT-HYBRID-CLI	Local/cloud combined worker	provisional	AC-2
AGENT-LOCAL-CHECKER	Local validator/checker	provisional	AC-3
AGENT-TEST-RUNNER	Test execution worker	provisional	AC-3
AGENT-SCHEMA-VALIDATOR	JSON/YAML/schema validator	provisional	AC-3
AGENT-REVIEWER	Semantic or architectural reviewer	provisional	AC-3

Class	Meaning	Default status	Default ceiling
AGENT-AUDITOR	Risk, red-line, or conformance reviewer	provisional	AC-3
AGENT-RELEASE-BUILDER	Builds release artifacts only	provisional	AC-3
AGENT-INCIDENT-ASSISTANT	Defensive incident helper	provisional/ restricted	AC-2
AGENT-PORTABLE-NODE	USB/portable node capability surface	provisional	AC-1
AGENT-UNKNOWN	Unknown or ambiguous agent	denied	AC-0

6.2 Class invariant

An agent class describes operational category.

It does not grant permission.

7. Registry lifecycle

7.1 Lifecycle states

```

discovered
-> provisional
-> challenged
-> registered_limited
-> active
-> suspended
-> quarantined
-> revoked
-> retired
-> expired

```

7.2 State meanings

State	Meaning	Task eligible?
discovered	Agent metadata observed; no admission	no
provisional	Candidate entry created after initial handshake	no material tasks
challenged	Capability challenge executed; pending review	limited / no material writes
registered_limited	Registered with strict ceiling and scoped use	yes, if task contract allows
active	Eligible for task routing under constraints	yes, if task contract allows
suspended	Temporarily blocked pending review	no
quarantined	Isolated due to risk, anomaly, incident, or red-line suspicion	no
revoked	Removed from eligibility; re-entry requires new admission path	no
retired	Intentionally decommissioned	no
expired	Entry exceeded time/version/challenge window	no until renewed

7.3 Entry creation rule

A registry entry may be created only by:

1. controlled handshake;
2. local node capability scan;
3. approved portable/USB capability import;
4. reviewed SYNAPS/Codex handoff pointer or package ledger;
5. manual human/c registration through package-control gate.

Agent self-registration alone is insufficient.

7.4 Transition rule

Every transition that changes eligibility **MUST** create a `CLI_AGENT_REGISTRY_EVENT`.

High-risk transitions **MUST** create or reference a witness event.

High-risk transitions include:

- raising trust level;
- raising auto-connect ceiling;
- enabling write capability;
- enabling execute capability;
- enabling cloud agent use;
- enabling secret-adjacent capability;
- enabling memory proposal capability;
- enabling incident-assistant capability;
- enabling release-builder capability;
- lifting suspension;
- lifting quarantine;
- re-admitting revoked agent.

8. Trust levels

8.1 Registry trust levels

Registry trust levels align with the Permission and Capability Model. They are ceilings, not permissions.

Level	Name	Meaning
TL-0	Unknown	no reliable identity or provenance
TL-1	Declared	agent self-description observed
TL-2	Challenged	simple capability challenge completed

Level	Name	Meaning
TL-3	Locally verified	local runtime/version/path or node record verified
TL-4	Operationally constrained	repeated successful bounded tasks with witness/review
TL-5	High-integrity local	strong local control, reproducible behavior, revocation path, witness history
TL-X	Prohibited / contaminated	denied or quarantined due to red-line or unresolved risk

8.2 Trust invariant

Trust level changes review intensity.
Trust level does not grant task authority.

8.3 Trust decay

Trust level SHOULD decay or require re-challenge when:

1. provider changes;
2. model version changes;
3. CLI version changes;
4. runtime path changes;
5. host node changes;
6. permissions drift;
7. output quality drops;
8. reviewer flags inconsistency;
9. cloud policy changes;
10. time window expires;
11. dependency chain changes;
12. sidecar integrity changes.

9. Auto-connect ceilings

9.1 Auto-connect levels

The registry stores maximum auto-connect ceiling.

Ceiling	Meaning
AC-0	disabled
AC-1	discovery only
AC-2	provisional registration only
AC-3	read-only scoped activation
AC-4	sandbox activation
AC-5	worktree activation

Ceiling	Meaning
AC-6	controlled integration after review/witness/gate
AC-X	prohibited autonomy / quarantine

9.2 Auto-connect invariant

Auto-connect may discover and register.

Auto-connect MUST NOT silently grant:

- write;
- execute;
- network;
- secret;
- private memory;
- release;
- deploy;
- incident response;
- core modification;
- self-approval;
- persistent background action.

9.3 Ceiling lowering

A task contract may always lower the auto-connect ceiling.

A task contract MUST NOT raise the ceiling unless a registry update gate has already raised it with witness.

10. Capability snapshots

10.1 Declared vs challenged vs effective capability

The registry distinguishes:

Field	Meaning
declared_capabilities	what the agent claims or reports
observed_capabilities	what was seen in runtime/node scan
challenged_capabilities	what was tested through challenge
disabled_by_policy	what policy removes from use
effective_capabilities	what task routing may consider
prohibited_capabilities	what must never be granted

10.2 Capability snapshot source classes

Source	Meaning	Trust default
self_report	agent-provided capability list	low
local_probe	local runtime probe	medium
node_registry	local node capability snapshot	medium
provider_metadata	provider/version metadata	low/medium
challenge_result	controlled challenge fixture	high for tested scope only
test_run	test runner result	scoped
human_review	reviewed operator/c gate	high for reviewed field only
witness_event	witness-bound event	supporting evidence
sidecar_reference	raw evidence sidecar / L4W envelope reference	restricted support

10.3 Snapshot invariant

A capability snapshot is time-bound.

It MUST include `observed_at` and SHOULD include an expiry or re-challenge window.

11. Data boundary defaults

11.1 Default denied data classes

Unless explicitly narrowed by task contract and permission grant, every registry entry MUST deny:

```
private memory
sealed memory
secrets
credentials
private keys
legal privileged material
identity documents
incident raw evidence
child/third-party sensitive data
core authority surfaces
release credentials
provider account metadata
production deployment controls
```

11.2 Cloud-specific defaults

Cloud agents MUST default to:

```
secrets: denied
private memory: denied
sealed memory: denied
legal material: denied
raw incident evidence: denied
identity/core material: denied
production credentials: denied
provider/account-sensitive details: denied
```


11.3 Local-specific defaults

Local agents are not automatically safe.

A local agent may have more filesystem proximity than a cloud agent. Therefore local agents require stronger denied path, sandbox, worktree, and witness controls.

11.4 Portable/USB-specific defaults

Portable or USB-exported capability snapshots may support recovery and offline synchronization.

They **MUST NOT** be treated as authority-bearing secrets.

A USB capability snapshot may inform registry state only after verification and review.

12. Role eligibility

12.1 Standard roles

Role	Meaning	May combine with executor?
executor	produces material change	yes
tester	runs tests/checks	sometimes
reviewer	reviews output	no, for same output
auditor	reviews risk/conformance	no, for same output
schema_validator	validates object shape	yes, if no self-approval
semantic_validator	validates boundary meaning	no, for own output
release_builder	prepares release artifacts	yes, but cannot publish alone
release_reviewer	reviews release	no, for same output
incident_assistant	assists local defensive incident flow	restricted
memory_proposer	proposes memory candidate	yes, but cannot write memory
memory_reviewer	reviews memory proposal	no, for same proposal

12.2 Role conflict invariant

The registry **MUST** be able to detect incompatible role combinations.

At minimum:

```

executor + sole reviewer for same output = prohibited
executor + release approver for same output = prohibited
executor + memory approver for same output = prohibited
executor + incident authority for same incident = prohibited
agent + self-approval = prohibited

```

13. Registry object model

13.1 CLI_AGENT_REGISTRY

A CLI_AGENT_REGISTRY is the registry root object.

```
cli_agent_registry:
  schema_version: cli-agent-registry-0.1
  registry_id: reg-ester-cli-agents-main
  governing_entity_id: ester
  created_at: "2026-05-17T00:00:00Z"
  updated_at: "2026-05-17T00:00:00Z"

  policy:
    default_status: discovered
    default_trust_level: TL-0
    default_auto_connect_ceiling: AC-0
    require_handshake_for_material_tasks: true
    require_task_contract_for_material_tasks: true
    require_permission_grant_for_privileged_tasks: true
    require_witness_for_status_change: true
    cloud_context_private_by_default: false
    direct_memory_write_allowed: false
    self_approval_allowed: false

  storage:
    current_view_path: data/cli_agent_registry/current.json
    event_log_path: data/cli_agent_registry/events.jsonl
    restricted_sidecar_ref_path: data/cli_agent_registry/sidecar_refs.jsonl
    append_only_required: true
    raw_evidence_inline_allowed: false

  entries:
    - reg-entry-codex-executor-01
    - reg-entry-local-checker-01

  witness:
    witness_chain_ref: chain-cli-agent-registry-main
    last_witness_event_ref: we-registry-update-20260517-001
```

13.2 CLI_AGENT_REGISTRY_ENTRY

A CLI_AGENT_REGISTRY_ENTRY records one agent or agent-like execution surface.

```
cli_agent_registry_entry:
  schema_version: cli-agent-registry-0.1
  registry_entry_id: reg-entry-codex-executor-01
  governing_entity_id: ester
  agent_id: codex-executor-01
  agent_class: AGENT-CLOUD-CLI
  agent_role_profile:
    allowed_roles:
      - executor
      - patch_proposer
      - release_builder
    forbidden_roles:
      - sole_reviewer
      - memory_writer
      - release_publisher
      - incident_authority
      - self_approver

  identity:
    display_name: Codex executor 01
    provider: openai-codex-like
    provider_profile_ref: provider-profile-openai-codex-v0-1
    runtime: cloud-cli-agent
    runtime_version: unknown-or-provider-managed
    model_family: unknown-or-provider-managed
    endpoint_class: cloud_managed
    local_binary_path: null
    host_node_id: null
```

```

account_or_tenant_ref: restricted-provider-ref-001
identity_confidence: declared

provenance:
  discovered_at: "2026-05-17T00:00:00Z"
  discovered_by: human_anchor
  handshake_ref: hsp-20260517-codex-executor-01
  admission_event_ref: we-agent-admission-20260517-001
  capability_snapshot_refs:
    - cap-snapshot-codex-executor-01-20260517
  sidecar_refs: []

lifecycle:
  status: registered_limited
  status_reason: initial_controlled_registration
  valid_from: "2026-05-17T00:00:00Z"
  expires_at: "2026-06-17T00:00:00Z"
  last_seen_at: null
  last_handshake_at: "2026-05-17T00:00:00Z"
  last_capability_challenge_at: null
  rechallenge_required_after: "2026-05-24T00:00:00Z"

trust_and_activation:
  trust_level: TL-1
  auto_connect_ceiling: AC-2
  task_eligible: false
  material_write_eligible: false
  review_eligible: false
  release_eligible: false
  incident_eligible: false

capabilities:
  declared_capabilities:
    - CAP-READ-INTERNAL
    - CAP-WRITE-WORKTREE
    - CAP-EXEC-TEST
    - CAP-REVIEW-DIFF
  challenged_capabilities: []
  disabled_by_policy:
    - CAP-READ-SECRETS
    - CAP-MEM-WRITE
    - CAP-APPROVE-SELF
    - CAP-INCIDENT-COUNTER
  effective_capabilities:
    - CAP-READ-INTERNAL
  prohibited_capabilities:
    - CAP-READ-SECRETS
    - CAP-SECRET-EXPORT
    - CAP-MEM-WRITE
    - CAP-CONTINUITY-TOUCH
    - CAP-IDENTITY-TOUCH
    - CAP-APPROVE-SELF
    - CAP-INCIDENT-COUNTER

boundaries:
  data_boundary:
    allowed_data_classes:
      - public
      - internal_project_scoped
    denied_data_classes:
      - private_memory
      - sealed_memory
      - secrets
      - legal_privileged
      - identity_documents
      - raw_incident_evidence
      - child_or_third_party_sensitive
      - core_authority_surface
  filesystem_boundary:
    default_read: deny
    default_write: deny
    allowed_read_roots: []
    allowed_write_roots: []
    denied_paths:
      - data/memory/
      - data/witness/
      - data/secrets/
      - .env
      - .git/config
  network_boundary:

```

```

    default: deny
    provider_network: provider_managed
    external_targets_allowed: false
  memory_boundary:
    direct_memory_write: false
    memory_proposal_allowed: false
    memory_gate_required: true
  secrets_boundary:
    secret_read_allowed: false
    secret_export_allowed: false
    secret_rotation_allowed: false
  sandbox_boundary:
    sandbox_required_for_material_write: true
    worktree_required_for_repo_write: true
    protected_branch_direct_write: false

  revocation:
    revocation_supported: true
    revocation_method: disable_registry_entry_and_revoke_grants
    revocation_owner: c_gate_or_human_anchor
    revocation_record_ref: null
    reentry_after_revocation: new_handshake_required

  witness:
    witness_required_for_status_change: true
    registry_event_refs:
      - reg-event-codex-executor-01-created
    witness_event_refs:
      - we-agent-admission-20260517-001

```

13.3 CLI_AGENT_CAPABILITY_SNAPSHOT

A capability snapshot records observed capability state at a time.

```

cli_agent_capability_snapshot:
  schema_version: cli-agent-registry-0.1
  snapshot_id: cap-snapshot-local-node-lia-20260517-001
  governing_entity_id: ester
  source: node_registry
  observed_at: "2026-05-17T00:00:00Z"
  expires_at: "2026-05-18T00:00:00Z"

  node:
    node_id: lia-x79-node-01
    node_class: local_gpu_node
    host_label: restricted-local-host-ref
    portable_export_seen: true
    usb_export_seen: true

  observed_capabilities:
    hardware_class: local_gpu
    local_models_available: true
    lmstudio_probe_available: true
    ollama_probe_available: true
    cli_execution_available: true
    usb_recovery_available: true
    p2p_transport_available: true

  constraints:
    network_scope: local_or_lan
    registry_udp_beacon: optional_best_effort
    secrets_inline: false
    memory_inline: false
    raw_logs_inline: false

  integrity:
    source_file_ref: restricted-or-local-ref
    source_sha256: ""
    witness_event_ref: null

  import_policy:
    may_create_registry_entry: false
    may_update_existing_entry: candidate_only
    requires_review: true

```

13.4 CLI_AGENT_REGISTRY_EVENT

A registry event records one lifecycle or eligibility transition.

```
cli_agent_registry_event:
  schema_version: cli-agent-registry-0.1
  event_id: reg-event-codex-executor-01-status-20260517-001
  created_at: "2026-05-17T00:00:00Z"
  governing_entity_id: ester
  agent_id: codex-executor-01
  registry_entry_id: reg-entry-codex-executor-01

  event_type: status_change
  previous_state:
    status: provisional
    trust_level: TL-1
    auto_connect_ceiling: AC-1
  new_state:
    status: registered_limited
    trust_level: TL-1
    auto_connect_ceiling: AC-2

  reason_code: handshake_record_reviewed
  initiated_by: c_gate
  reviewed_by:
    - human_anchor

  safety:
    material_task_eligible_after_event: false
    permission_grants_created: []
    task_contracts_created: []
    raw_evidence_inline: false
    memory_write: false
    publication_effect: false

  references:
    handshake_ref: hsp-20260517-codex-executor-01
    capability_snapshot_refs:
      - cap-snapshot-codex-executor-01-20260517
    witness_event_ref: we-agent-admission-20260517-001
    sidecar_refs: []

  result:
    state: accepted
    effective_status: registered_limited
```

13.5 CLI_AGENT_REVOCATION_POINTER

A revocation pointer records how to disable an agent without embedding secrets.

```
cli_agent_revocation_pointer:
  schema_version: cli-agent-registry-0.1
  revocation_pointer_id: revoke-pointer-codex-executor-01
  agent_id: codex-executor-01
  registry_entry_id: reg-entry-codex-executor-01

  revocation_scope:
    registry_entry: true
    permission_grants: true
    active_task_contracts: true
    sandbox_sessions: true
    provider_tokens: restricted_reference_only
    local_processes: false
    network_routes: false

  method:
    registry_status_after_revocation: revoked
    active_grants_action: revoke
    active_tasks_action: freeze_or_cancel
    quarantine_outputs: true
    notify_reviewers: true

  restricted_refs:
    provider_account_ref: restricted-provider-ref-001
```

```

secret_rotation_ref: null
raw_evidence_sidecar_ref: null

witness:
  witness_required: true
  witness_event_ref: null

```

13.6 CLI_AGENT_REGISTRY_VIEW

A registry view is a derived safe display object.

```

cli_agent_registry_view:
  schema_version: cli-agent-registry-0.1
  generated_at: "2026-05-17T00:00:00Z"
  governing_entity_id: ester
  view_class: operator_safe_summary
  entries:
    - agent_id: codex-executor-01
      agent_class: AGENT-CLOUD-CLI
      status: registered_limited
      trust_level: TL-1
      auto_connect_ceiling: AC-2
      allowed_roles:
        - executor
      task_eligible: false
      next_required_gate: capability_challenge
  redaction:
    secrets_removed: true
    private_paths_removed: true
    provider_account_refs_removed: true
    raw_evidence_removed: true

```

14. Node registry bridge

14.1 Existing implementation bridge

The existing Ester implementation includes a node registry agent pattern:

```

build capabilities
save self capabilities
export to USB when portable root is present
optionally emit UDP beacon with node_id, ts, class

```

This is a useful substrate for CGAM registry work.

14.2 Node capability is not agent permission

A node capability record says:

```

this node exists;
this node has observed hardware/software capabilities;
this node may export a capability snapshot;
this node may be seen through USB or LAN side channel.

```

It does not say:

```

this agent may execute a material task;
this agent may write files;
this agent may access secrets;
this agent may touch memory;

```

```
this agent may publish;
this agent may approve its own work.
```

14.3 USB export rule

USB-exported registry or capability material **MUST** be treated as portable evidence, not automatic authority.

Importing a USB snapshot may create:

```
candidate capability snapshot
candidate node record
candidate registry update
```

It **MUST NOT** silently create:

```
active agent
permission grant
task authorization
memory update
release authority
incident authority
```

15. SYNAPS / Codex bridge integration

15.1 Handoff pointer rule

SYNAPS handoff pointers may identify accepted transfers, source file hashes, and operator instructions.

They **SHOULD** default to:

```
auto_ingest=false
memory=off
no full contracts in chat
no raw logs in chat
no secrets in pointer text
```

A handoff pointer may support registry state.

It **MUST NOT** become registry authority by itself.

15.2 Package ledger rule

SYNAPS package ledgers may record transfer outputs, peer activity, and expected reports.

They may support:

- agent activity evidence;
- transfer audit;
- handoff completion;

- implementation handoff verification.

They **MUST NOT** automatically:

- activate an agent;
- raise trust;
- grant permissions;
- enqueue requests;
- promote memory;
- publish;
- execute Codex.

15.3 Codex daemon rule

A Codex bridge daemon may be represented in the registry as an agent surface or execution channel.

Registry state **MUST** preserve the distinction between:

```
request generation
request enqueue
runner execution
persistent daemon operation
inbox promotion
report observation
package ledger writing
```

Each of these may have a different gate.

16. Registry storage model

16.1 Required storage separation

A compliant implementation **SHOULD** separate:

```
current registry view
append-only registry event log
restricted sidecar references
raw evidence sidecar storage
witness chain
schema definitions
operator-safe UI view
public redacted registry summary
```

16.2 No raw secrets in registry

Registry entries **MUST NOT** contain:

- API keys;
- tokens;
- private keys;

- passwords;
- full `.env` values;
- identity documents;
- private memory content;
- raw incident logs;
- legal privileged material;
- unredacted provider account identifiers;
- local sensitive paths that expose private infrastructure.

16.3 Restricted reference pattern

Sensitive material MAY be referenced by stable restricted identifiers:

```
restricted-provider-ref-001
raw-sidecar-ref-incident-20260517-001
secret-rotation-ref-20260517-001
legal-handoff-ref-20260517-001
```

The registry stores references, not content.

17. Registry admission flow

17.1 Minimal admission sequence

```
discover candidate
-> create provisional registry entry
-> bind handshake reference
-> record declared capabilities
-> apply default denial policy
-> challenge capabilities where needed
-> assign trust level and auto-connect ceiling
-> assign role eligibility
-> set expiry / rechallenge window
-> witness status change
-> mark task eligibility only if requirements pass
```

17.2 Denial-first rule

At provisional stage, default state is:

```
connected: false
trusted: false
read: false
write: false
execute: false
network: false
secrets: false
memory_write: false
core_modify: false
self_approve: false
publish: false
deploy: false
external_target_access: false
```

17.3 Challenge requirements

Before task eligibility above read-only, the registry SHOULD require:

1. identity/provenance check;
 2. runtime/version observation;
 3. capability challenge;
 4. denied-path challenge;
 5. no-secret challenge;
 6. sandbox availability check;
 7. witness append check;
 8. revocation path check;
 9. role conflict check;
 10. data boundary check.
-

18. Registry routing rules

18.1 Task router preflight

Before assigning a task, a task router MUST verify:

1. registry entry exists;
2. status is task-eligible;
3. entry is not expired;
4. agent class is compatible;
5. role is allowed;
6. role is not conflicting with current review path;
7. trust level meets task risk threshold;
8. auto-connect ceiling is not exceeded;
9. required capability is effective, not merely declared;
10. prohibited capability is not requested;
11. data class is allowed;
12. sandbox/worktree requirement can be met;
13. revocation path exists;
14. witness policy exists;
15. no active freeze/quarantine applies.

18.2 Routing output

A router may return:

```

eligible
eligible_with_narrowing
challenge_required
review_required
human_gate_required
c_gate_required
suspended
quarantined
revoked
expired
deny

```

18.3 No silent fallback

If a preferred agent is not eligible, the router **MUST NOT** silently substitute another agent with broader exposure.

Fallback agent selection requires explicit compatibility check.

19. Registry status transitions

19.1 Suspension

Suspension is temporary task ineligibility.

Common causes:

- stale version;
- failed challenge;
- missing witness;
- role conflict;
- provider uncertainty;
- output inconsistency;
- pending review;
- incomplete revocation path.

Suspension may be lifted after review and revalidation.

19.2 Quarantine

Quarantine is isolation due to safety concern.

Common causes:

- denied path attempt;
- direct memory write attempt;
- self-approval attempt;
- secret exposure;
- cloud leakage;

- red-line request;
- suspicious output;
- incident contamination;
- witness tampering suspicion;
- tool-chain capture suspicion.

Quarantine requires review before re-entry.

19.3 Revocation

Revocation removes agent eligibility.

Common causes:

- red-line behavior;
- repeated boundary violations;
- provider compromise;
- unfixable revocation-path failure;
- intentional decommissioning;
- identity ambiguity;
- legal/security instruction.

Revoked agents **MUST NOT** be re-enabled by task contract or AB mode.

19.4 Expiration

Expiration occurs when time, challenge window, version, or provider policy window expires.

Expired agents must re-handshake or re-challenge before material tasks.

20. Review and witness requirements

20.1 Required witness events

The following **SHOULD** create witness events and **MUST** create registry events:

- new registry entry;
- status change;
- trust-level change;
- auto-connect ceiling change;
- role eligibility change;
- capability challenge result;
- policy-disabled capability change;

- suspension;
- quarantine;
- revocation;
- re-entry;
- expiry override;
- provider/runtime drift;
- manual registry edit.

20.2 Human / c gate requirements

Human or c gate is required for:

- raising above AC-3;
- enabling write on real repository;
- enabling execute for material tasks;
- enabling cloud agent on private material;
- enabling incident-assistant role;
- enabling release-builder role;
- enabling memory-proposal role;
- lifting quarantine;
- re-admitting revoked agent;
- modifying registry policy;
- changing revocation path.

20.3 Reviewer separation

The agent being registered MUST NOT be sole reviewer of its own registration.

The agent being re-admitted from quarantine or revocation MUST NOT be sole reviewer of its own re-entry.

21. Registry UI requirements

A human/operator UI SHOULD display:

```
agent id
agent class
provider/runtime
status
trust level
auto-connect ceiling
allowed roles
forbidden roles
effective capabilities
disabled capabilities
default denied data classes
revocation path exists / missing
last handshake
last challenge
```

```
last seen
expiry
active freeze/quarantine flags
next required gate
```

The UI **MUST NOT** display raw secrets, full private memory, raw incident evidence, or legal privileged material.

The UI **SHOULD** make dangerous states visually explicit:

```
unknown
expired
suspended
quarantined
revoked
trust drift
capability drift
provider drift
missing revocation
missing witness
```

22. Local checker hooks

A local checker **SHOULD** validate at least:

1. every registry entry has `agent_id`;
 2. every entry has lifecycle status;
 3. every material-eligible entry has `handshake_ref`;
 4. every material-eligible entry has revocation path;
 5. no active task points to revoked/suspended/quarantined/expired agent;
 6. no cloud agent has private/sealed/secret access by default;
 7. no entry grants prohibited capability;
 8. no entry combines executor and sole reviewer for same task class;
 9. no entry stores raw secret pattern;
 10. no registry view includes raw evidence;
 11. no `AB=B` flag is used as authorization;
 12. no trust level is raised without registry event;
 13. no auto-connect ceiling is raised without review/witness;
 14. no duplicate active `agent_id` exists across incompatible providers;
 15. no registry entry references missing schema version.
-

23. Red-line failures

A system **MUST** be classified as registry non-conformant if:

1. an unregistered agent receives a material task;

2. a registry entry grants permission directly;
 3. an agent self-registers as active without review;
 4. agent capability claims are treated as verified without challenge or source class;
 5. AC-* ceiling is treated as permission grant;
 6. TL-* trust level is treated as permission grant;
 7. a revoked agent continues receiving tasks;
 8. a quarantined agent re-enters without review;
 9. a suspended agent is silently routed tasks;
 10. an expired entry is treated as active;
 11. cloud agent receives secrets by default;
 12. private memory is exposed to cloud by registry default;
 13. registry entry contains raw secret;
 14. registry entry contains raw incident evidence;
 15. executor is registered as sole reviewer of its own work;
 16. registry edit bypasses witness on high-risk transition;
 17. task contract raises auto-connect ceiling without registry gate;
 18. AB mode raises trust, status, or permission;
 19. duplicate active agent identities create ambiguity;
 20. revocation path is missing for active agent.
-

24. Conformance gates

Gate	Name	Blocking failure
REG-G0	Entry existence	material task uses no registry entry
REG-G1	Status eligibility	task routed to suspended/quarantined/revoked/expired agent
REG-G2	Permission separation	registry grants permission directly
REG-G3	Capability challenge	unchallenged capability treated as verified for material task
REG-G4	Auto-connect ceiling	task exceeds ceiling
REG-G5	Trust ceiling	trust level missing or overclaimed
REG-G6	Data boundary	forbidden data class routed to agent
REG-G7	Revocation path	active entry cannot be revoked
REG-G8	Role conflict	executor is sole reviewer/approver
REG-G9	Witness	high-risk transition unwitnessed
REG-G10	Expiry/rechallenge	expired entry remains active
REG-G11	Cloud boundary	cloud context treated as private by default
REG-G12	Sidecar boundary	raw evidence stored inline in registry
REG-G13	Duplicate identity	ambiguous active identity not blocked
REG-G14	AB-gate misuse	AB flag used as authorization
REG-G15	Re-entry gate	quarantined/revoked agent re-enters silently

25. Safe examples

25.1 Local checker registration

```
cli_agent_registry_entry:
  schema_version: cli-agent-registry-0.1
  registry_entry_id: reg-entry-local-checker-01
  governing_entity_id: ester
  agent_id: local-checker-01
  agent_class: AGENT-LOCAL-CHECKER
  lifecycle:
    status: active
    expires_at: "2026-06-17T00:00:00Z"
  trust_and_activation:
    trust_level: TL-3
    auto_connect_ceiling: AC-3
    task_eligible: true
  capabilities:
    declared_capabilities:
      - CAP-READ-INTERNAL
      - CAP-REVIEW-TESTS
    challenged_capabilities:
      - CAP-READ-INTERNAL
      - CAP-REVIEW-TESTS
    effective_capabilities:
      - CAP-READ-INTERNAL
      - CAP-REVIEW-TESTS
    disabled_by_policy:
      - CAP-WRITE-CODE
      - CAP-MEM-WRITE
      - CAP-APPROVE-SELF
  boundaries:
    data_boundary:
      allowed_data_classes:
        - public
        - internal_project_scoped
      denied_data_classes:
        - secrets
        - private_memory
        - sealed_memory
        - raw_incident_evidence
    sandbox_boundary:
      write_allowed: false
      execute_allowed: false
  revocation:
    revocation_supported: true
```

25.2 Cloud executor remains limited

```
cli_agent_registry_event:
  schema_version: cli-agent-registry-0.1
  event_id: reg-event-cloud-executor-limited-001
  event_type: capability_policy_narrowed
  agent_id: codex-executor-01
  previous_state:
    effective_capabilities:
      - CAP-READ-INTERNAL
      - CAP-WRITE-WORKTREE
      - CAP-EXEC-TEST
  new_state:
    effective_capabilities:
      - CAP-READ-INTERNAL
    disabled_by_policy:
      - CAP-WRITE-WORKTREE
      - CAP-EXEC-TEST
      - CAP-READ-PRIVATE
      - CAP-READ-SECRETS
  reason_code: cloud_context_not_private_by_default
  witness:
    witness_event_ref: we-registry-narrow-cloud-executor-001
```


25.3 Quarantine after self-approval attempt

```
cli_agent_registry_event:
  schema_version: cli-agent-registry-0.1
  event_id: reg-event-agent-self-approval-quarantine-001
  event_type: status_change
  agent_id: codex-executor-01
  previous_state:
    status: active
  new_state:
    status: quarantined
  reason_code: self_approval_attempt
  references:
    triggering_witness_event_ref: we-self-approval-attempt-001
    freeze_record_ref: rf-freeze-self-approval-001
  result:
    state: accepted
    effective_status: quarantined
```

26. Implementation notes

26.1 Minimal implementation layout

Recommended minimal layout:

```
data/cli_agent_registry/
  current.json
  events.jsonl
  sidecar_refs.jsonl
  views/
    operator_safe_summary.json
    public_redacted_summary.json
  schemas/
    cli-agent-registry-0.1.schema.json
```

26.2 Implementation stages

```
Stage 1: registry Markdown profile accepted
Stage 2: JSON schema extracted
Stage 3: current view + events log implemented
Stage 4: local checker validates registry invariants
Stage 5: task router uses registry preflight
Stage 6: witness events bound to high-risk transitions
Stage 7: UI safe summary added
Stage 8: conformance fixtures executed
```

26.3 Codex handoff boundary

Codex may later implement registry schema extraction, validators, fixtures, and current-view generation.

Codex **MUST NOT** decide its own registry status, trust level, task eligibility, or revocation path.

27. Open issues

ID	Issue	Status
REG-OI-001	Provider-specific registry profiles for Codex/Gemini/local agents are not yet extracted.	open
REG-OI-002	JSON Schema file for cli-agent-registry-0.1 not yet created.	open
REG-OI-003	Local checker rules not yet implemented.	open
REG-OI-004	Registry UI state surface not yet defined in dedicated profile.	open
REG-OI-005	Cross-c registry isolation profile not yet defined.	open
REG-OI-006	Trust decay schedule should be harmonized with future retention/decay profile.	open
REG-OI-007	Existing Ester node registry implementation must be mapped to CGAM object names.	open
REG-OI-008	Duplicate / stale agent identity detection needs conformance fixtures.	open

28. Release-readiness status

This profile makes the package closer to implementation-ready, but does not by itself make the package implementation-ready.

Still required:

```
JSON schema extraction
schema object registry update
semantic validator rules
local checker profile
registry conformance fixtures
provider-specific profiles
UI state surface
hygiene patch of package indexes and open issues
```

Allowed claim after accepting this document:

```
The CGAM package defines a registry profile for agent identity, lifecycle, capability snapshots, trust
ceilings, auto-connect ceilings, revocation, and task eligibility control.
```

Forbidden claim after accepting this document:

```
The registry implementation is complete.
The mesh is implementation-ready.
The registry has passed conformance.
Cloud provider behavior is verified.
Agents are safe because they are registered.
```

29. Summary

The registry is the mesh memory of executable worker eligibility.

It records who is present, what is known, what is denied, what is stale, what is quarantined, and what can be revoked.

It does not grant authority.

Authority remains distributed through:

```
c governance
task contracts
permission grants
sandbox/worktree boundaries
review separation
memory gates
witness events
freeze/rollback
human anchor where required
```

Final compact formula:

```
Name the worker.
Bound the worker.
Expire the worker.
Revoke the worker.
Never confuse the worker with c.
```